

EDGE WEATHER

Smart Environmental Monitoring Station

Project Name	Office Environmental IoT Node
Hardware Core	ESP32-S3-WROOM-1U
Documentation Type	Combined Technical Reference & Firmware Manual
Release Version	v1.4 (Assembled & Tested)
Author	Systems Engineering Team

Note: Wi-Fi and Firebase credentials in the firmware appendix have been replaced with placeholders for safe sharing. Insert your real values locally before flashing the device.

Table of Contents

Section	Page
1. Executive Summary	3
2. System Architecture & Pinout Mapping	3
3. Hardware Components — Part A	4
4. Hardware Components — Part B	4
5. Non-Blocking Firmware Design	5
6. Firebase Realtime Database Synchronization	5
7. Centered OLED Layouts & Navigation	6
8. Operation, Calibration & Debugging	6
Appendix A: Full Firmware Source Code	7

1. Executive Summary

The EDGE WEATHER environmental monitoring node is a specialized, production-ready IoT device engineered specifically for indoor office climates. It stands out from typical IoT implementations by using a strict **Local-First** execution architecture.

Even if local corporate Wi-Fi connections drop, access to the Firebase cloud is lost, or database tokens expire, the physical device never stalls. Sensor polling, localized OLED updates, edge-triggered system alerts, capacitive touch handling, and PIR standby operations execute seamlessly on the microcontroller. Cloud syncing resumes automatically in the background once network handshakes are successfully completed.

2. System Architecture & Bus Design

The central computing unit of the EDGE WEATHER node is the ESP32-S3-WROOM-1U SoC. It connects to the environmental sensor suite and OLED display using a shared single I2C bus routed through physical GPIO pins 11 (SDA) and 12 (SCL). Peripheral interrupts and analog reads use direct designated pins to eliminate bus overhead.

Pinout Mapping Table

Peripheral / Signal	ESP32-S3 GPIO Pin	Connection Type	Description
Shared I2C SDA	GPIO 11	Digital (Shared)	Data line for OLED, HTU21D, and BMP180
Shared I2C SCL	GPIO 12	Digital (Shared)	Clock line for OLED, HTU21D, and BMP180
LDR Input	GPIO 13	Analog Input	Reads ambient light voltage divider
TOUCH_PIN	GPIO 45	Digital Input	Signal output of the TTP223 capacitive touch pad
RAIN_PIN	GPIO 48	Digital Input	Digital output of the MH-RD Rain Sensor
PIR_PIN	GPIO 9	Digital Input	Signal output of the HC-SR501 motion sensor

3. Core Components (Part A)

ESP32-S3 Microcontroller

Runs on dual-core Xtensa 32-bit LX7 processors. Manages non-blocking multitasking logic, schedules Wi-Fi connections, and executes real-time sensor processing.

0.96" SSD1306 OLED Display

A high-contrast 128x64 pixel screen. Communicates via I2C at address 0x3C. Features a custom-rendered menu and edge-triggered alerts.

HTU21D Humidity & Temp Sensor

Provides digital temperature and relative humidity outputs. Communicates over I2C to deliver measurements with typical accuracies of $\pm 0.3^{\circ}\text{C}$ and $\pm 2\%$ RH.

4. Core Components (Part B)

BMP180 Pressure Sensor

High-accuracy barometric pressure sensor. Delivers pressure readings in Pa, which are converted to hPa in the firmware to track changing weather states.

LDR Ambient Light Sensor

Wired in an inverted voltage divider circuit. High ADC values correspond to dark conditions, while low ADC values signify bright ambient light.

MH-RD Rain Sensor

Uses a digital comparator to output a LOW state when raindrops bridge its trace board, triggering high-priority rain interrupts on GPIO 48.

TTP223 Touch Interface

A capacitive touch module. Outputs a clean HIGH level when touched. Triggers the screen to wake and switch pages instantly.

HC-SR501 PIR Motion Sensor

Uses passive infrared detection to monitor occupancy. Immediately wakes the OLED from sleep mode when motion is detected nearby.

5. Non-Blocking Firmware Design

To prevent database read/write timeouts or connection delays from locking the local interface, the firmware runs on a completely cooperative multitasking state machine. No `delay()` calls are used; execution loops run asynchronously using `millis()` timer comparisons.

```
void loop() {
  // Asynchronously execute Firebase network queues
  if (firebaseInitialized && WiFi.status() == WL_CONNECTED) {
    Firebase.ready();
  }
  // Poll environmental metrics on a 2-second non-blocking timer
  processSensors();
  // Evaluate state boundaries and prioritize alert overlays
  checkAlertConditions();
  handleAlertTimeout();
  // Check Touch inputs and PIR presence to control sleep modes
  processInputs();
  // Update display values only if state or parameter values change
  updateDisplay();
  // Background non-blocking network reconnect loop
  handleWiFi();
  handleFirebaseUpload();
}
```

6. Firebase RTDB Synchronization

The edge station synchronizes localized climate variables with a Firebase Realtime Database (RTDB). To prevent database flooding and conserve network bandwidth, the firmware implements **Delta Synchronization**. Values are written to Firebase only when they differ from their previously uploaded states.

Additionally, to recover from any potential network packet loss during transitions, the firmware runs a Force Sync Timer every 30 seconds. This guarantees that the database and web dashboard stay fully in sync with the physical sensors.

RTDB Schema

```
/weather_station
|-- temperature: 24.5
|-- humidity: 60
|-- pressure: 1013
|-- light_status: "Normal"
|-- is_raining: false
`-- online: true
```

7. Centered OLED Layouts & Navigation

The display engine features a custom horizontal centering function that aligns parameters in the dead center of the screen, providing a polished look on the 0.96-inch OLED display. To keep track of navigation, a smartphone-style page dot array is drawn at the bottom of the screen. The active page is represented by a filled dot, while inactive pages are hollow. Display text wrapping is disabled completely to prevent text from wrapping, clipping, or overlapping during alerts.

Centering Function

```
void printCentered(const char* text, int y, int size) {
    int charWidth = 6 * size; // Character width at size
    int len = strlen(text);
    int x = (SCREEN_WIDTH - (len * charWidth)) / 2;
    if (x < 0) x = 0;
    display.setTextSize(size);
    display.setCursor(x, y);
    display.print(text);
}
```

8. Operation, Calibration & Debugging

The edge station is designed to configure itself seamlessly. When powered, the startup screen runs an animated dot sequence while initializing sensors, then turns OFF to conserve power. Standby is bypassed immediately upon PIR motion detection or physical touch.

Diagnostic Troubleshooting Table

Symptom	Root Cause	Action Required
Firebase Token Error	NTP clock is not synchronized. SSL authentication fails because the ESP32 clock is at 1970.	Verify local network internet access and confirm NTP UDP port 123 is open.
Reversed LDR	Analog voltage divider is wired backwards (LDR on the high side).	The firmware handles this automatically by inverting the readings (4095 - ldrEMA). No wiring changes needed.
OLED stays OFF	Display is in Standby mode. Standby activates after 15 seconds of inactivity.	Wave a hand in front of the PIR sensor or tap the touch button to wake.
OLED is flickering	display.clearDisplay() is called repeatedly inside the fast main loop.	The firmware prevents this by updating values only on state changes or clock ticks.

Appendix A: Full Firmware Source Code

The complete production firmware for the ESP32-S3-WROOM-1U is listed below. **Wi-Fi and Firebase credentials have been replaced with placeholders** — substitute your own values in a local copy before compiling and flashing.

```
/**
 * =====
 *          EDGE WEATHER - IOT WEATHER STATION
 *    Production Firmware for ESP32-S3-WROOM-1U Microcontroller
 * =====
 *
 * Hardware Layout & I2C Bus Sharing:
 * - OLED SSD1306, HTU21D, and BMP180 share the same I2C bus (Pins 11 and 12).
 * - TTP223 capacitive touch sensor handles non-blocking screen navigation.
 * - HC-SR501 PIR motion sensor manages OLED standby/wake-on-approach.
 * - LDR analog sensor monitors ambient illumination levels with filtering.
 * - MH-RD rain sensor triggers priority precipitation alerts.
 */

#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <Adafruit_HTU21DF.h>
#include <Adafruit_BMP085.h>
#include <WiFi.h>
#include "time.h"

// Firebase Core and RTDB Libraries
#include <Firebase_ESP_Client.h>
#include <addons/TokenHelper.h>
#include <addons/RTDBHelper.h>

/*
 * =====
 * 1. CONFIGURATION & CREDENTIALS
 * =====
 */
#define WIFI_SSID          "A70"
#define WIFI_PASSWORD      "YOUR_WIFI_PASSWORD"

#define FIREBASE_API_KEY   "YOUR_FIREBASE_API_KEY"
#define FIREBASE_DB_URL    "https://wifiweather-e2dd0-default-rtdb.asia-southeast1.firebaseio.com"
#define FIREBASE_USER_EMAIL "YOUR_FIREBASE_USER_EMAIL"
#define FIREBASE_USER_PASS "YOUR_FIREBASE_USER_PASSWORD"

// NTP Time Synchronization Details
const char* ntpServer      = "pool.ntp.org";
const long  gmtoffset_sec  = 19800; // Timezone Offset (UTC +5:30 = 19800s. Adjust for your region)
const int   daylightOffset_sec = 0; // Daylight Savings Offset in seconds

/*
 * =====
 * 2. PIN DEFINITIONS
 * =====
 */
#define I2C_SDA            10
#define I2C_SCL            11
#define LDR_PIN            13
#define RAIN_PIN           48
#define TOUCH_PIN          45
#define PIR_PIN            9

/*
 * =====
 * 3. GLOBAL VARIABLES & STATE MANAGEMENT
 * =====
 */
// OLED Dimensions
#define SCREEN_WIDTH       128
#define SCREEN_HEIGHT      64
#define OLED_RESET         -1

// Hardware Class Instantiations
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
```

```

Adafruit_HTU21DF htu = Adafruit_HTU21DF();
Adafruit_BMP085 bmp;

// Firebase Handler Objects
FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;
bool firebaseInitialized = false;

// Local Sensor Variables (Filtered)
float temperature = 0.0;
int humidity = 0;
int pressure = 0;
String light_status = "Normal";
bool isRaining = false;

// Last Uploaded Data (To satisfy the requirement of only uploading changes)
float lastUploadedTemp = -99.9;
int lastUploadedHum = -1;
int lastUploadedPress = -1;
String lastUploadedLight = "";
int lastUploadedRain = -1;
bool lastUploadedOnline = false;

// UI and Navigation Management
int currentPage = 0; // Pages: 0=Temp, 1=Hum, 2=Press, 3=Light, 4=Rain
int lastDrawnPage = -1;
int lastTimeSec = -1;
bool displayOn = false; // Managed by sleep timer and PIR sensor
unsigned long lastUserActivityTime = 0;
const unsigned long oledSleepTimeout = 15000; // 15-second visual timeout

// Debouncing state
bool touchActive = false;

// Alert Logic State Machine
bool alertActive = false;
int activeAlertPriority = 0; // 0 = None, 1 = Rain, 2 = Temp, 3 = Hum, 4 = Press, 5 = Light
String alertTitle = "";
String alertValue = "";
unsigned long alertStartTime = 0;
unsigned long alertDuration = 0;
int normalPageBeforeAlert = 0;
bool lastAlertState = false;

// Transition flags to prevent continuous re-triggering of alerts while conditions are met
bool rainAlertTriggered = false;
bool tempAlertTriggered = false;
bool humAlertTriggered = false;
bool pressAlertTriggered = false;
bool lightAlertTriggered = false;

// Network & Event Timers (Non-blocking)
unsigned long lastSensorReadTime = 0;
const unsigned long sensorReadInterval = 2000; // Read sensors every 2 seconds

unsigned long lastWifiCheckTime = 0;
const unsigned long wifiCheckInterval = 10000; // Check Wi-Fi state every 10 seconds

unsigned long lastFirebaseCheckTime = 0;
const unsigned long firebaseCheckInterval = 3000; // Try uploading changes every 3 seconds

unsigned long lastFullSyncTime = 0;
const unsigned long fullSyncInterval = 30000; // Full DB push every 30 seconds to guarantee consistency

// Filtering Variables
float ldrEMA = 0.0;
const float ldrFilterAlpha = 0.15; // Smooth filtering coefficient

/*
 * =====
 * 4. SYSTEM DECLARATIONS
 * =====
 */
void wakeDisplay();
void triggerAlert(int priority, const char* title, const char* valueStr, unsigned long durationMs,
    int targetPage);
void handleAlertTimeout();
bool isTimeSynced();
void setupFirebase();
void printCentered(const char* text, int y, int size);
void printCenteredAuto(const char* text, int y, int defaultSize);
void drawPageIndicator();

```

```

void showBootScreen();

/*
 * =====
 * 5. SENSOR MANAGER MODULE
 * =====
 */
void processSensors() {
    if (millis() - lastSensorReadTime < sensorReadInterval) {
        return;
    }
    lastSensorReadTime = millis();

    // 1. Read HTU21D (Temperature & Humidity)
    float rawTemp = htu.readTemperature();
    float rawHum = htu.readHumidity();

    if (!isnan(rawTemp) && rawTemp > -40.0 && rawTemp < 125.0) {
        temperature = rawTemp;
    }
    if (!isnan(rawHum) && rawHum >= 0.0 && rawHum <= 100.0) {
        humidity = (int)rawHum;
    }

    // 2. Read BMP180 (Atmospheric Pressure)
    float rawPress = bmp.readPressure();
    if (rawPress > 0) {
        pressure = (int)(rawPress / 100.0); // Convert Pa to hPa
    }

    // 3. Read & Filter LDR (Ambient Light Level)
    int rawLDR = analogRead(LDR_PIN);
    if (ldrEMA == 0.0) {
        ldrEMA = rawLDR;
    }
    ldrEMA = (ldrFilterAlpha * rawLDR) + ((1.0 - ldrFilterAlpha) * ldrEMA);

    // Invert the filtered value to correct reversed LDR divider hardware (dark = high reading, brig
    ht = low reading)
    float invertedLDR = 4095.0 - ldrEMA;

    // Categorize inverted LDR value
    if (invertedLDR < 400) {
        light_status = "Dark";
    } else if (invertedLDR < 1200) {
        light_status = "Dim";
    } else if (invertedLDR < 2400) {
        light_status = "Normal";
    } else if (invertedLDR < 3400) {
        light_status = "Bright";
    } else {
        light_status = "Very Bright";
    }

    // 4. Read Rain Sensor
    bool rawRainState = (digitalRead(RAIN_PIN) == LOW);
    isRaining = rawRainState;

    // Print system telemetry to Serial Monitor
    Serial.println("--- Telemetry Update ---");
    Serial.print("Temp: "); Serial.print(temperature, 1); Serial.println(" C");
    Serial.print("Hum: "); Serial.print(humidity); Serial.println(" %");
    Serial.print("Pres: "); Serial.print(pressure); Serial.println(" hPa");
    Serial.print("Light: "); Serial.println(light_status);
    Serial.print("Rain: "); Serial.println(isRaining ? "Raining" : "No Rain");
    Serial.print("Time Synced: "); Serial.println(isTimeSynced() ? "Yes" : "No");
    Serial.print("Free Heap: "); Serial.print(ESP.getFreeHeap()); Serial.println(" bytes");
    Serial.println("-----");
}

/*
 * =====
 * 6. ALERT MANAGER MODULE
 * =====
 */
void checkAlertConditions() {
    // Priority 1: Rain Detection (Edge-triggered)
    if (isRaining) {
        if (!rainAlertTriggered) {
            triggerAlert(1, "RAIN DETECTED", "Raining", 10000, 4);
            rainAlertTriggered = true;
        }
    } else {
        rainAlertTriggered = false;
    }
}

```

```

}

// Priority 2: Critical High Temperature (> 38C) (Edge-triggered)
if (temperature > 38.0) {
    if (!tempAlertTriggered) {
        char tempBuf[16];
        dtostrf(temperature, 2, 1, tempBuf);
        strcat(tempBuf, " C");
        triggerAlert(2, "HIGH TEMPERATURE", tempBuf, 8000, 0);
        tempAlertTriggered = true;
    }
    else {
        tempAlertTriggered = false;
    }
}

// Priority 3: High Humidity (> 85%) (Edge-triggered)
if (humidity > 85) {
    if (!humAlertTriggered) {
        char humBuf[16];
        itoa(humidity, humBuf, 10);
        strcat(humBuf, " %");
        triggerAlert(3, "HIGH HUMIDITY", humBuf, 8000, 1);
        humAlertTriggered = true;
    }
    else {
        humAlertTriggered = false;
    }
}

// Priority 4: Atmospheric Pressure Anomalies (< 980 hPa or > 1035 hPa) (Edge-triggered)
bool outOfBoundsPress = (pressure < 980 || pressure > 1035);
if (outOfBoundsPress) {
    if (!pressAlertTriggered) {
        if (pressure < 980) {
            char pressBuf[16];
            itoa(pressure, pressBuf, 10);
            strcat(pressBuf, " hPa");
            triggerAlert(4, "LOW PRESSURE ALERT", pressBuf, 8000, 2);
        }
        else {
            char pressBuf[16];
            itoa(pressure, pressBuf, 10);
            strcat(pressBuf, " hPa");
            triggerAlert(4, "HIGH PRESSURE ALERT", pressBuf, 8000, 2);
        }
        pressAlertTriggered = true;
    }
    else {
        pressAlertTriggered = false;
    }
}

// Priority 5: Ambient Dark/Low light condition (Edge-triggered)
bool isLowLight = (light_status == "Dark" || light_status == "Dim");
if (isLowLight) {
    if (!lightAlertTriggered) {
        triggerAlert(5, "LOW LIGHT WARNING", light_status.c_str(), 5000, 3);
        lightAlertTriggered = true;
    }
    else {
        lightAlertTriggered = false;
    }
}
}

void triggerAlert(int priority, const char* title, const char* valueStr, unsigned long durationMs,
    int targetPage) {
    if (!alertActive || priority < activeAlertPriority) {
        if (!alertActive) {
            normalPageBeforeAlert = currentPage;
        }
        activeAlertPriority = priority;
        alertTitle = title;
        alertValue = valueStr;
        alertDuration = durationMs;
        alertStartTime = millis();
        alertActive = true;
        currentPage = targetPage;
        wakeDisplay();
        Serial.printf("ALERT TRIGGERED [Priority %d]: %s (%s)\n", priority, title, valueStr);
    }
}

void handleAlertTimeout() {
    if (alertActive) {
        if (millis() - alertStartTime >= alertDuration) {
            Serial.println("Alert display completed. Returning to normal monitoring.");
            alertActive = false;
        }
    }
}

```

```

        activeAlertPriority = 0;
        currentPage = normalPageBeforeAlert;
        lastUserActivityTime = millis();
    }
}

/*
 * =====
 * 7. DISPLAY MANAGER MODULE
 * =====
 */
/**
 * Renders a string centered horizontally on the OLED screen.
 */
void printCentered(const char* text, int y, int size) {
    int charWidth = 6 * size;
    int len = strlen(text);
    int x = (SCREEN_WIDTH - (len * charWidth)) / 2;
    if (x < 0) x = 0;
    display.setTextSize(size);
    display.setCursor(x, y);
    display.print(text);
}

/**
 * Centered text drawing that automatically scales down font size if the
 * text is wider than the display width to avoid horizontal cutting/clipping.
 */
void printCenteredAuto(const char* text, int y, int defaultSize) {
    int size = defaultSize;
    int len = strlen(text);
    while (size > 1 && (len * 6 * size) > (SCREEN_WIDTH - 12)) {
        size--; // Decrease font size progressively until the string fits
    }
    printCentered(text, y, size);
}

/**
 * Smartphone-style home screen dot indicator drawn at the bottom of the display.
 */
void drawPageIndicator() {
    int dotRadius = 2;
    int spacing = 8;
    int totalWidth = (5 * 2 * dotRadius) + (4 * spacing);
    int startX = (SCREEN_WIDTH - totalWidth) / 2;
    int y = 58;

    for (int i = 0; i < 5; i++) {
        int x = startX + i * (2 * dotRadius + spacing) + dotRadius;
        if (i == currentPage) {
            display.fillCircle(x, y, dotRadius, SSD1306_WHITE); // Active page (filled dot)
        } else {
            display.drawCircle(x, y, dotRadius, SSD1306_WHITE); // Inactive page (hollow dot)
        }
    }
}

/**
 * Renders a startup load screen animation on boot.
 */
void showBootScreen() {
    display.ssd1306_command(SSD1306_DISPLAYON);
    displayOn = true;

    // Run the loading dot sequence for 6 cycles (~2.4 seconds total)
    for (int cycle = 0; cycle < 6; cycle++) {
        display.clearDisplay();
        display.setTextColor(SSD1306_WHITE);

        // Clean, centered modern titles
        printCentered("EDGE", 14, 2);
        printCentered("WEATHER", 30, 2);

        // Loading dots styling
        int dotY = 52;
        int dotRadius = 3;
        int spacing = 12;
        int totalDotsWidth = (3 * 2 * dotRadius) + (2 * spacing);
        int startX = (SCREEN_WIDTH - totalDotsWidth) / 2;

        for (int i = 0; i < 3; i++) {
            int x = startX + i * (2 * dotRadius + spacing) + dotRadius;
            if (cycle % 3 == i) {

```

```

        display.fillCircle(x, dotY, dotRadius + 1, SSD1306_WHITE); // Bouncing filled active dot
    } else {
        display.drawCircle(x, dotY, dotRadius, SSD1306_WHITE); // Inactive hollow dot
    }
}

display.display();
delay(400); // Standard microsecond blocking ONLY during startup boot sequence
}

// Clear display and return to standby OFF
display.clearDisplay();
display.display();
display.ssd1306_command(SSD1306_DISPLAYOFF);
displayOn = false;
Serial.println("Boot screen complete. Standby Mode enabled.");
}

void renderOLED() {
    display.clearDisplay();
    display.setTextColor(SSD1306_WHITE);

    // 1. Unified Clean Top Status Bar
    display.setTextSize(1);
    display.setCursor(4, 2);
    display.print("EDGE WEATHER");

    struct tm timeinfo;
    char timeBuf[12] = "--:--:--";
    // CRITICAL FIX: Timeout passed as 0ms to prevent blocking the CPU when offline
    if (getLocalTime(&timeinfo, 0)) {
        sprintf(timeBuf, "%02d:%02d:%02d", timeinfo.tm_hour, timeinfo.tm_min, timeinfo.tm_sec);
    }
    display.setCursor(80, 2);
    display.print(timeBuf);

    // Clean line divider
    display.drawFastHLine(0, 11, SCREEN_WIDTH, SSD1306_WHITE);

    // 2. Main Area Drawing
    if (alertActive) {
        // Double industrial alert border frame to prevent any text bleed
        display.drawRect(4, 15, 120, 46, SSD1306_WHITE);
        display.drawRect(6, 17, 116, 42, SSD1306_WHITE);

        // Auto-scaled centered alert parameters to avoid screen clipping
        printCenteredAuto(alertTitle.c_str(), 22, 1);
        printCenteredAuto(alertValue.c_str(), 36, 2);
    } else {
        // Professional device page layout (Scoped with braces to prevent compiler jump errors)
        switch (currentPage) {
            case 0: { // Temperature Display
                printCentered("TEMPERATURE", 16, 1);
                char tempStr[16];
                dtostrf(temperature, 2, 1, tempStr);

                // C-style string formatting to bypass ambiguous String summation operations
                char tempFormatted[24];
                snprintf(tempFormatted, sizeof(tempFormatted), "%s°C", tempStr, (char)248);

                printCentered(tempFormatted, 28, 3);
                drawPageIndicator();
                break;
            }

            case 1: { // Humidity Display
                printCentered("HUMIDITY", 16, 1);
                char humStr[16];
                itoa(humidity, humStr, 10);
                strcat(humStr, "%");
                printCentered(humStr, 28, 3);
                drawPageIndicator();
                break;
            }

            case 2: { // Barometric Pressure Display
                printCentered("BAROMETRIC PRESSURE", 16, 1);
                char pressStr[16];
                itoa(pressure, pressStr, 10);
                strcat(pressStr, " hPa");
                printCentered(pressStr, 28, 2);
                drawPageIndicator();
                break;
            }
        }
    }
}

```

```

        case 3: { // Ambient Light Level Display
            printCentered("LIGHT LEVEL", 16, 1);
            printCenteredAuto(light_status.c_str(), 28, 2);
            drawPageIndicator();
            break;
        }

        case 4: { // Rain Precipitation Display
            printCentered("PRECIPITATION", 16, 1);
            printCentered(isRaining ? "RAIN ACTIVE" : "NO RAIN", 28, 2);
            drawPageIndicator();
            break;
        }
    }
}
display.display();
}

void updateDisplay() {
    if (!displayOn) {
        return;
    }

    struct tm timeinfo;
    int currentSec = -1;
    // CRITICAL FIX: Timeout passed as 0ms to prevent blocking the CPU when offline
    if (getLocalTime(&timeinfo, 0)) {
        currentSec = timeinfo.tm_sec;
    }

    bool redrawNeeded = false;
    if (currentSec != lastTimeSec) {
        lastTimeSec = currentSec;
        redrawNeeded = true;
    }
    if (currentPage != lastDrawnPage) {
        lastDrawnPage = currentPage;
        redrawNeeded = true;
    }
    if (alertActive != lastAlertState) {
        lastAlertState = alertActive;
        redrawNeeded = true;
    }

    if (redrawNeeded) {
        renderOLED();
    }
}

void sleepDisplay() {
    if (displayOn) {
        display.clearDisplay();
        display.display();
        display.ssd1306_command(SSD1306_DISPLAYOFF);
        displayOn = false;
        alertActive = false;
        activeAlertPriority = 0;
        Serial.println("Display Standby Activated. (OLED Off)");
    }
}

void wakeDisplay() {
    if (!displayOn) {
        display.ssd1306_command(SSD1306_DISPLAYON);
        displayOn = true;
        lastDrawnPage = -1;
        lastTimeSec = -1;
        Serial.println("Display Wake Detected. (OLED On)");
    }
    lastUserActivityTime = millis();
}

/*
 * =====
 * 8. TOUCH & PIR INPUT MODULE
 * =====
 */

void processInputs() {
    bool motionDetected = (digitalRead(PIR_PIN) == HIGH);
    if (motionDetected) {
        if (!displayOn) {
            Serial.println("PIR Event: Motion detected.");
            wakeDisplay();
        } else {

```

```

        lastUserActivityTime = millis();
    }
}

// Read raw touch sensor state
bool rawTouchState = (digitalRead(TOUCH_PIN) == HIGH);
static unsigned long lastTouchTime = 0;
const unsigned long touchCooldown = 350; // Lockout window prevents double-triggering and bounce

// Instant response debouncing: fires on first connection, then locks out bounce
if (rawTouchState == HIGH) {
    if (millis() - lastTouchTime > touchCooldown) {
        lastTouchTime = millis();
        lastUserActivityTime = millis();
        wakeDisplay();

        if (!alertActive) {
            currentPage = (currentPage + 1) % 5;
            Serial.printf("Touch Event: Page switched to %d.\n", currentPage);
        }
    }
}

if (displayOn && (millis() - lastUserActivityTime > oledSleepTimeout)) {
    sleepDisplay();
}
}

/*
 * =====
 * 9. WI-FI & TIME MANAGER MODULE
 * =====
 */
void handleWiFi() {
    static bool firstInit = true;

    if (firstInit) {
        Serial.println("Initializing network stack...");
        WiFi.mode(WIFI_STA);
        WiFi.setAutoReconnect(true); // Let ESP32 background stack handle it silently without blocking
        WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
        firstInit = false;
        return;
    }

    // Clean clock synchronization triggered asynchronously only on active transitions
    static bool lastConnectedState = false;
    bool currentConnectedState = (WiFi.status() == WL_CONNECTED);

    if (currentConnectedState && !lastConnectedState) {
        Serial.println("Network status: Online. Synchronizing NTP clock...");
        configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);
    }
    lastConnectedState = currentConnectedState;
}

bool isTimeSynced() {
    struct tm timeinfo;
    // CRITICAL FIX: Timeout passed as 0ms to prevent blocking the CPU when offline
    if (!getLocalTime(&timeinfo, 0)) {
        return false;
    }
    return (timeinfo.tm_year > 70);
}

/*
 * =====
 * 10. FIREBASE MANAGER MODULE
 * =====
 */
void handleFirebaseUpload() {
    if (WiFi.status() != WL_CONNECTED) {
        return;
    }

    // ONLY initialize Firebase after Network and NTP Time synchronization are verified.
    if (!firebaseInitialized) {
        if (isTimeSynced()) {
            Serial.println("Network connected and NTP time verified. Initializing Firebase...");
            setupFirebase();
            firebaseInitialized = true;
        } else {
            return;
        }
    }
}

```

```

}

if (!Firebase.ready()) {
  return;
}

// Periodic Force Sync to ensure website recovers if packet loss occurs
bool forceSync = false;
if (millis() - lastFullSyncTime >= fullSyncInterval) {
  lastFullSyncTime = millis();
  forceSync = true;
  Serial.println("Firebase: Initiating scheduled 30-second full status synchronization...");
}

if (!forceSync && (millis() - lastFirebaseCheckTime < firebaseCheckInterval)) {
  return;
}
lastFirebaseCheckTime = millis();

// 1. Send active online status flag
if (forceSync || !lastUploadedOnline) {
  if (Firebase.RTDB.setBoolAsync(&fbdo, "/weather_station/online", true)) {
    lastUploadedOnline = true;
  }
}

// 2. Temperature update
float tempRounded = round(temperature * 10.0) / 10.0;
if (forceSync || (abs(tempRounded - lastUploadedTemp) >= 0.1)) {
  if (Firebase.RTDB.setDoubleAsync(&fbdo, "/weather_station/temperature", tempRounded)) {
    lastUploadedTemp = tempRounded;
    Serial.println("Firebase upload: Temperature synchronized.");
  }
}

// 3. Humidity update
if (forceSync || (humidity != lastUploadedHum)) {
  if (Firebase.RTDB.setIntAsync(&fbdo, "/weather_station/humidity", humidity)) {
    lastUploadedHum = humidity;
    Serial.println("Firebase upload: Humidity synchronized.");
  }
}

// 4. Pressure update
if (forceSync || (pressure != lastUploadedPress)) {
  if (Firebase.RTDB.setIntAsync(&fbdo, "/weather_station/pressure", pressure)) {
    lastUploadedPress = pressure;
    Serial.println("Firebase upload: Pressure synchronized.");
  }
}

// 5. Light Level update (Critical for website rendering)
if (forceSync || (light_status != lastUploadedLight)) {
  if (Firebase.RTDB.setStringAsync(&fbdo, "/weather_station/light_status", light_status)) {
    lastUploadedLight = light_status;
    Serial.println("Firebase upload: Light Status synchronized.");
  }
}

// 6. Rain Status update (Critical for website rendering)
if (forceSync || (isRaining != lastUploadedRain)) {
  if (Firebase.RTDB.setBoolAsync(&fbdo, "/weather_station/is_raining", isRaining)) {
    lastUploadedRain = isRaining;
    Serial.println("Firebase upload: Rain Status synchronized.");
  }
}
}

void setupFirebase() {
  config.api_key = FIREBASE_API_KEY;
  config.database_url = FIREBASE_DB_URL;

  auth.user.email = FIREBASE_USER_EMAIL;
  auth.user.password = FIREBASE_USER_PASS;

  config.token_status_callback = tokenStatusCallback;

  Firebase.begin(&config, &auth);

  // CRITICAL FIX: Disabled to prevent the library from taking blocking control of the Wi-Fi stack
  Firebase.reconnectWiFi(false);

  Serial.println("Firebase service engine configured.");
}

```

```

/*
 * =====
 * 11. MAIN SETUP & SYSTEM INITIALIZATION
 * =====
 */
void setup() {
  Serial.begin(115200);
  while (!Serial && millis() < 3000) {
    ;
  }
  Serial.println("\n--- EDGE WEATHER Startup Sequence ---");

  // Initialize hardware pins. Touch pin uses internal pulldown for electrical stability.
  pinMode(LDR_PIN, INPUT);
  pinMode(RAIN_PIN, INPUT_PULLUP);
  pinMode(TOUCH_PIN, INPUT_PULLDOWN);
  pinMode(PIR_PIN, INPUT);

  // Start I2C bus using Pin 11 (SDA) and Pin 12 (SCL)
  Wire.begin(I2C_SDA, I2C_SCL);

  // 1. Initialize Display
  if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println("CRITICAL: SSD1306 display allocation failed.");
  } else {
    // Disable automatic text wrapping to prevent vertical overlapping/cutting
    display.setTextWrap(false);

    // Play the Boot Screen Animation immediately on power-up
    showBootScreen();
  }

  // 2. Initialize HTU21D
  if (!htu.begin()) {
    Serial.println("CRITICAL: HTU21D humidity sensor missing.");
  } else {
    Serial.println("HTU21D initialized successfully.");
  }

  // 3. Initialize BMP180
  if (!bmp.begin()) {
    Serial.println("CRITICAL: BMP180 pressure sensor missing.");
  } else {
    Serial.println("BMP180 initialized successfully.");
  }

  // 4. Initial sensor processing to retrieve data values before alerts start
  processSensors();

  // 5. Connect network & queue NTP synchronization
  handleWiFi();
  configTime(gmtOffset_sec, daylightOffset_sec, ntpServer);

  Serial.println("System initialized. Awaiting network, time, and interactions.");
  lastUserActivityTime = millis();
}

/*
 * =====
 * 12. MAIN PROGRAM LOOP
 * =====
 */
void loop() {
  // CRITICAL FIX: Only call Firebase.ready() if WiFi is active and Firebase is fully initialized.
  // This keeps loop speeds fast and prevents timeouts when offline.
  if (firebaseInitialized && WiFi.status() == WL_CONNECTED) {
    Firebase.ready();
  }

  // Read measurements and process filters
  processSensors();

  // Monitor weather constraints and process active overlays
  checkAlertConditions();
  handleAlertTimeout();

  // Handle touch navigation and passive motion sensing
  processInputs();

  // Write screen buffer changes if display is awake
  updateDisplay();

  // Background network and cloud loops
  handleWiFi();
}

```

```
    handleFirebaseUpload();  
}
```